Name: Ronit Sahoo
Class: D20C
Roll No: 29
Batch: B

BLOCKCHAIN EXPERIMENT NO: 6

Aim - Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Theory:

1. Smart Contracts and the Ballot Contract

A smart contract is a program stored on a blockchain that executes automatically when predefined conditions are met. Once deployed, its code cannot be altered, and every state change it makes is recorded permanently on the ledger, which makes smart contracts well suited to applications where participants must be able to trust an outcome without trusting any central party.

The Ballot contract is the standard example of an electronic voting system on Ethereum. A chairperson deploys the contract with a list of proposals, grants voting rights to selected addresses, and those voters may then either cast their vote directly or delegate it to another voter they trust. Because the tally is maintained on-chain, anyone can verify the result, and the contract logic itself guarantees that no address can vote twice or vote without permission.

2. Relevance of require Statements in Solidity

In Solidity, the require statement acts as a guard condition within a function. It ensures that only valid inputs or authorized callers can execute the code that follows it. If the condition evaluates to false, execution stops immediately and every state change made during that transaction is reverted to its original state. This all-or-nothing rollback guarantees that an invalid transaction can never leave the contract in a partially updated, corrupted state.

In the Ballot contract, require is used to enforce every rule of the election:

- **Authorization -** only the chairperson may grant voting rights: require(msg.sender == chairperson, ...).
- **Eligibility -** a caller must hold voting weight before voting: require(sender.weight != 0, "Has no right to vote").
- **No double voting -** a voter who has already voted cannot vote again: require(!sender.voted, "Already voted.").
- **No duplicate rights -** voting rights cannot be granted twice to the same address.
- **Delegation safety -** self-delegation and circular delegation chains are rejected.

The optional second argument supplies an error message returned to the caller, which makes debugging and user interaction considerably easier. Unused gas is refunded when a require fails, unlike the older behaviour in which a failed assertion consumed all remaining gas.

3. The Keywords mapping, storage and memory

- **mapping -** a key-value data structure similar to a hash table, declared as mapping(keyType => valueType). In this contract, mapping(address => Voter) public voters associates every Ethereum address with a Voter structure. Mappings provide constant-time lookup but have no length and cannot be iterated over, and every possible key exists by default with a zero-initialised value, which is why an address that was never granted rights simply has a weight of 0.
- **storage -** the permanent data area of the contract, held on the blockchain itself. Contract-level state variables live in storage by default and persist across transactions. Writing to storage is the most expensive operation in terms of gas. Declaring a local variable as Voter storage

sender = voters[msg.sender] creates a reference to the stored record, so modifying sender modifies the on-chain data directly.

- **memory -** a temporary data area that exists only for the duration of a function call and is discarded afterwards. It is used for function arguments, return values and intermediate computations, and is far cheaper than storage. The constructor parameter string[] memory proposalNames and the return value string memory winnerName_ are both held in memory.

The distinction between storage and memory is not merely about cost. Assigning a storage value to a memory variable creates an independent copy, so changes to that copy would not be saved. Using storage references where state must be updated is therefore a matter of correctness as well as efficiency.

4. Why bytes32 Instead of string?

The original Ballot contract stores proposal names as bytes32 rather than string, and the reason is gas efficiency.

- **bytes32 -** a fixed-size value type that always occupies exactly 32 bytes and fits into a single 256-bit storage slot. Storage, copying and comparison are simple and cheap. Its limitation is that names are restricted to 32 bytes and must be supplied in hexadecimal form, which is inconvenient for users.
- **string -** a dynamically-sized type that can hold text of any length and can be entered directly as readable text. Because its length is variable, the EVM must store a length value plus the data across one or more slots, which increases the gas cost of storing and handling it.

In this experiment the contract was modified to use string for proposal names, as required by the task. Four changes were needed: the name field of the Proposal struct, the constructor parameter from bytes32[] memory to string[] memory, and the return type of winnerName from bytes32 to string memory. The result is that proposals can be supplied as readable names such as "Sandy" rather than as 32-byte hexadecimal values, at the cost of higher deployment and storage gas. Tools such as the Web3 Type Converter exist precisely because the bytes32 form is not human-readable.

5. Vote Delegation Logic

A voter may transfer their voting power to another voter. When delegate(to) is called, the contract first follows any existing delegation chain to its end and rejects the request if the chain loops back to the caller. It then marks the caller as having voted and applies one of two branches depending on the delegate's state. If the delegate has already voted, the caller's weight is added directly to the vote count of the proposal the delegate chose. If the delegate has not yet voted, the caller's weight is instead added to the delegate's own weight, so that when the delegate eventually votes, their single vote carries the combined weight.

Software Requirements:

- Remix IDE (browser based)
- Solidity compiler 0.8.34
- Remix VM (Prague) as the execution environment
- Ethers.js (bundled with Remix)
- Web3 Type Converter for bytes32 and string conversion

Tasks Performed:

1. Opened Remix IDE and navigated to the contracts folder under Workspaces.
2. Opened and studied the Ballot.sol contract to understand its logic.

3. Modified the proposal name type from bytes32 to string in the Proposal struct, the constructor and the winnerName function, saving the contract as Voting.sol.
4. Compiled the contract with Solidity compiler 0.8.34.
5. Deployed the contract from Account 1 with the proposal names "Sandy", "Ruturaj" and "Sumant".
6. Verified the chairperson address and inspected the stored proposals.
7. Granted voting rights from the chairperson to Accounts 2, 3 and 4.
8. Cast a vote from Account 2 and verified the updated vote count and voter record.
9. Tested the require guards by attempting a repeated vote and a duplicate grant of voting rights.
10. Delegated the vote of Account 3 to Account 4 and verified the transferred weight.
11. Queried winningProposal and winnerName to obtain the result.

Accounts Used:

Account	Address	Role
Account 1	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4	Chairperson (deployer)
Account 2	0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2	Voter - voted for Ruturaj
Account 3	0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db	Voter - delegated to Account 4
Account 4	0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB	Voter - received delegation

Program (Voting.sol):

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;
/**
* @title Voting
* @dev Implements voting process along with vote delegation.
* Proposal names changed from bytes32 to string.
*/
contract Voting {
// Represents a single voter.
struct Voter {
uint weight; // accumulated by delegation
bool voted; // if true, already voted
address delegate; // person delegated to
uint vote; // index of the voted proposal
}
// Represents a single proposal.
struct Proposal {
string name; // proposal name (was bytes32)
uint voteCount; // number of accumulated votes
}
address public chairperson;
// Stores a 'Voter' struct for each possible address.
mapping(address => Voter) public voters;
// A dynamically-sized array of 'Proposal' structs.
Proposal[] public proposals;
/**
* @dev Create a new ballot to choose one of 'proposalNames'.
* @param proposalNames names of proposals (string[])
*/
constructor(string[] memory proposalNames) {
chairperson = msg.sender;
voters[chairperson].weight = 1;
for (uint i = 0; i < proposalNames.length; i++) {
```

```
proposals.push(Proposal({ name:
proposalNames[i],
voteCount: 0
}));
}
}
/**
* @dev Give 'voter' the right to vote on this
ballot.
* May only be called by 'chairperson'.
*/
function giveRightToVote(address voter)
external {
require(
msg.sender == chairperson,
"Only chairperson can give right to vote."
);
require(
!voters[voter].voted,
"The voter already voted."
);
require(
voters[voter].weight == 0,
"Voter already has the right to vote."
);
voters[voter].weight = 1;
}
/**
* @dev Delegate your vote to the voter 'to'.
*/
function delegate(address to) external {
Voter storage sender = voters[msg.sender];
require(sender.weight != 0, "You have no
right to vote");
require(!sender.voted, "You already voted.");
require(to != msg.sender, "Self-delegation is
disallowed.");
// Forward the delegation as long as 'to' also
delegated.
while (voters[to].delegate != address(0)) {
to = voters[to].delegate;
require(to != msg.sender, "Found loop in
delegation.");
}

Voter storage delegate_ = voters[to];
// Voters cannot delegate to accounts that
cannot vote.
require(delegate_.weight >= 1);
sender.voted = true;
sender.delegate = to;
if (delegate_.voted) {
// If the delegate already voted,
// directly add to the number of votes.
proposals[delegate_.vote].voteCount +=
sender.weight;
} else {
// If the delegate did not vote yet,
// add to her weight.
delegate_.weight += sender.weight;
}
}
/**
* @dev Give your vote (including votes
delegated to you)
* to proposal 'proposals[proposal].name'.
*/
function vote(uint proposal) external {
Voter storage sender = voters[msg.sender];
require(sender.weight != 0, "Has no right to
vote");
require(!sender.voted, "Already voted.");
sender.voted = true;
sender.vote = proposal;
// Reverts automatically if 'proposal' is out of
range.
proposals[proposal].voteCount +=
sender.weight;
}
/**
* @dev Computes the winning proposal
index.
*/
function winningProposal()
public
view returns (uint winningProposal_)
```

```
{
uint winningVoteCount = 0;
for (
uint p = 0;
p < proposals.length;
p++
) {
if (proposals[p].voteCount >
winningVoteCount) {
winningVoteCount = proposals[p].voteCount;
winningProposal_ = p;
}
}
```

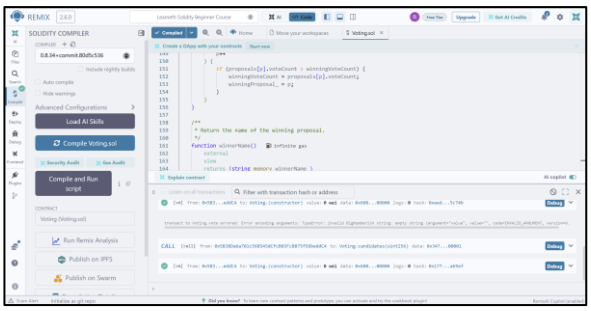
```
}
/**
* Return the name of the winning proposal.
*/
function winnerName()
external
view
returns (string memory winnerName_)
{
winnerName_ =
proposals[winningProposal()].name;
}
}
```

Output:

1. Compiling and Deploying the Contract

Voting.sol was compiled successfully with compiler 0.8.34. It was then deployed on the Remix VM from Account 1, with the proposal names supplied as a readable string array ["Sandy","Ruturaj","Sumant"], confirming that the bytes32 to string change was effective.

Output:



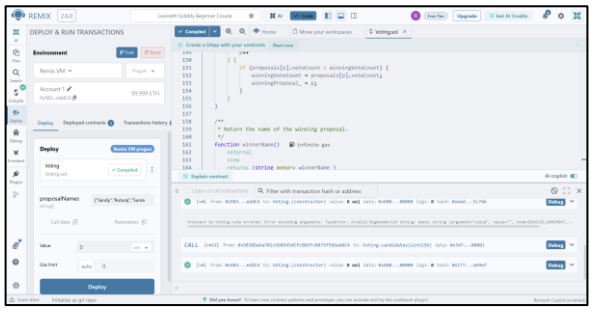


Fig 1 (left): Voting.sol compiled with Solidity 0.8.34. Fig 2 (right): Deployment with string proposal names from Account 1.

2. Verifying the Chairperson and Proposals

Calling chairperson returned 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4, the address of Account 1 that deployed the contract. Querying the proposals array showed each name stored as readable text with a vote count of zero.

Output:

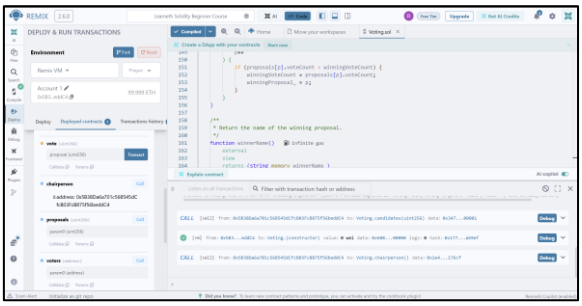
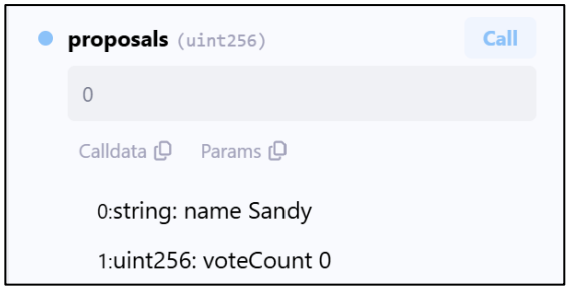


Fig 3: chairperson returns the address of Account 1, the deployer.



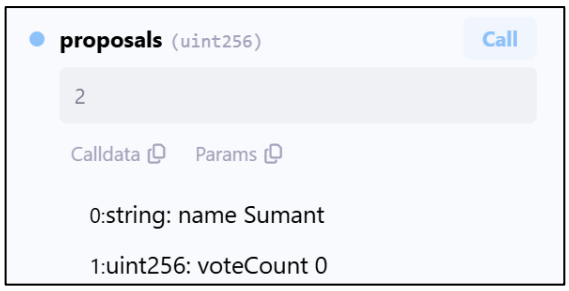
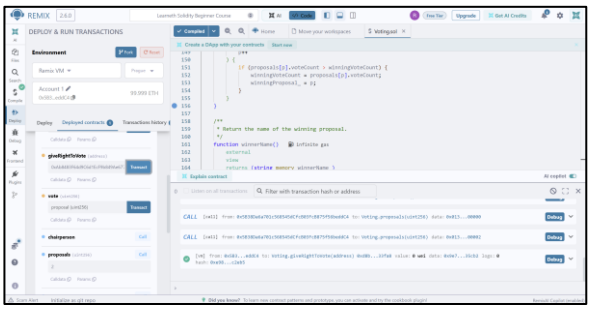


Fig 4 (left): proposals(0) returns Sandy with 0 votes. *Fig 5 (right): proposals(2) returns Sumant with 0 votes.*

3. Granting the Right to Vote and Casting a Vote

From Account 1, the chairperson called giveRightToVote with the address of Account 2. Switching to Account 2, a vote was then cast for proposal index 1. Both transactions succeeded.

Output:



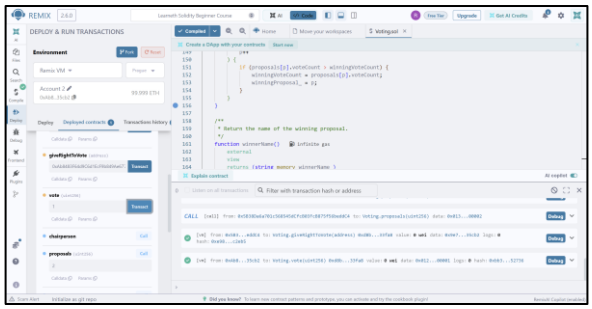
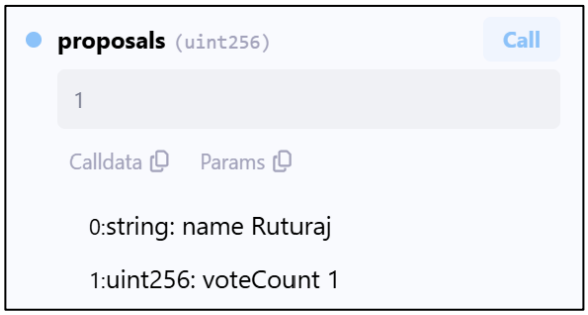


Fig 6 (left): Chairperson grants voting rights to Account 2. *Fig 7 (right): Account 2 votes for proposal index 1.*

4. Verifying the Vote

Proposal 1, Ruturaj, now showed a vote count of 1. The voter record for Account 2 showed a weight of 1, voted set to true, no delegate, and vote recorded as proposal 1.

Output:



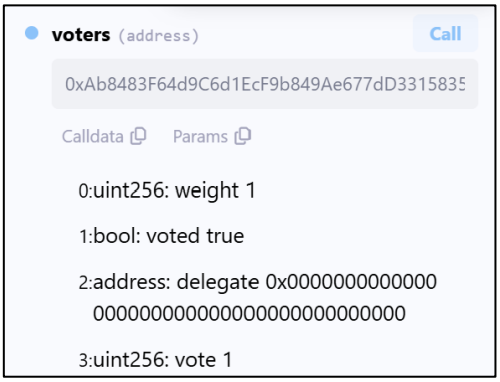


Fig 8 (left): Ruturaj now has 1 vote. *Fig 9 (right): Voter record of Account 2 after voting.*

5. require Guard - Preventing a Repeated Vote

Account 2 attempted to vote a second time, for proposal 0. The transaction was reverted with the reason "Already voted.", demonstrating that the require statement blocks double voting and rolls back all state changes.

Output:

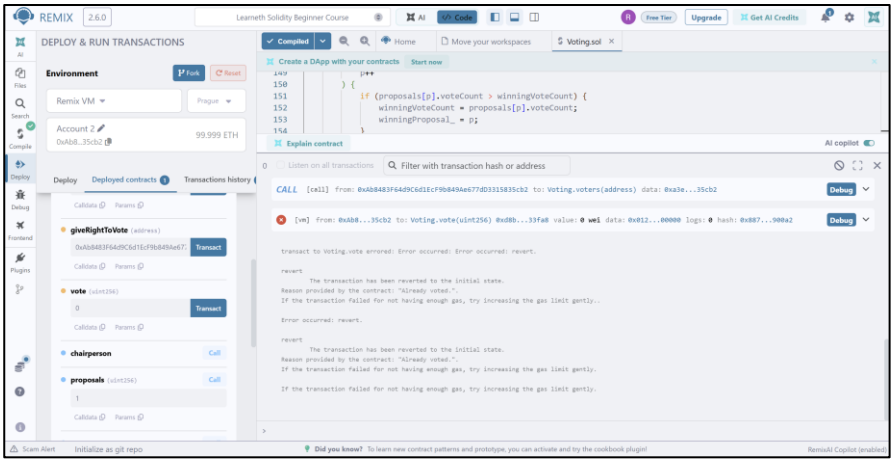
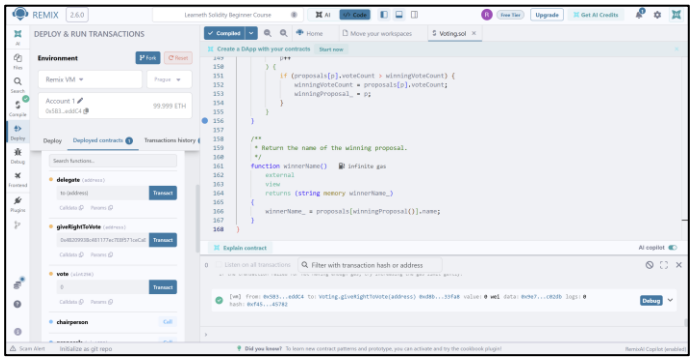


Fig 10: Second vote from Account 2 reverted with "Already voted."

6. Granting Voting Rights to Accounts 3 and 4

The chairperson granted voting rights to Account 3 and Account 4. Account 4's voter record confirmed a weight of 1 with voted still false.

Output:



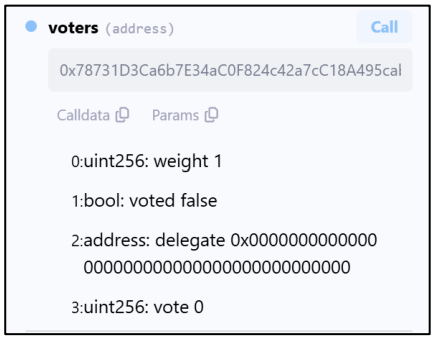


Fig 11 (left): Chairperson grants voting rights to Account 3. Fig 12 (right): Account 4 holds a weight of 1 and has not voted.

7. Delegating a Vote

An attempt to grant voting rights again to an address that already held them was reverted with "Voter already has the right to vote.". Account 3 then delegated its vote to Account 4, and the transaction succeeded.

Output:

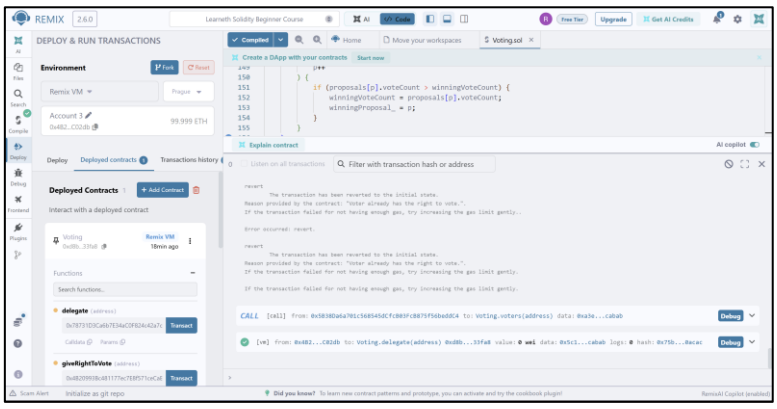
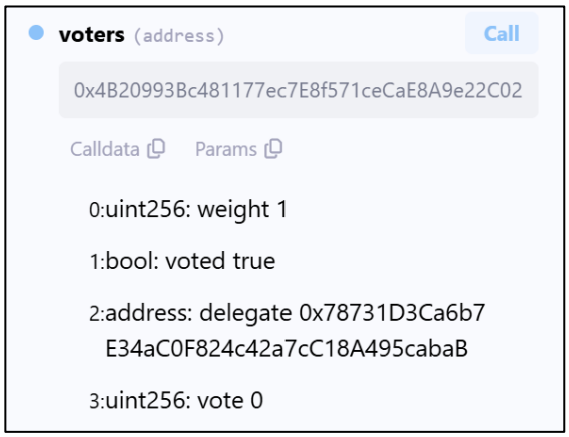


Fig 13: Duplicate grant reverted, followed by Account 3 delegating to Account 4.

After delegation, Account 3 was marked as voted with its delegate set to Account 4. Account 4's weight rose from 1 to 2, since Account 4 had not yet voted and the delegated weight was therefore added to its own.



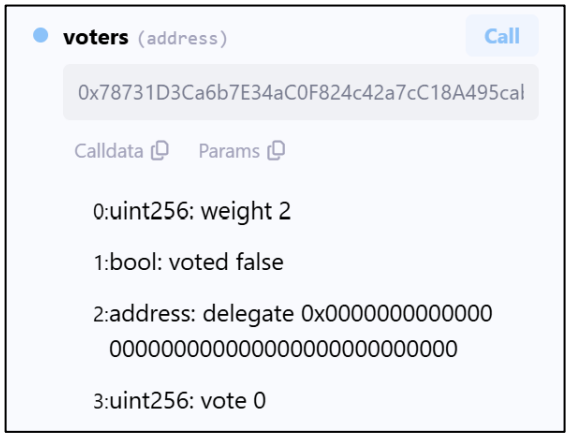
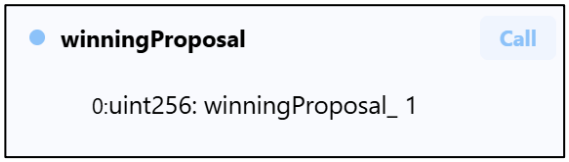


Fig 14 (left): Account 3 voted = true, delegate = Account 4. *Fig 15 (right): Account 4 weight increased from 1 to 2.*

8. Declaring the Winner

winningProposal returned index 1 and winnerName returned "Ruturaj" as a readable string, confirming that the modified return type worked as intended.

Output:



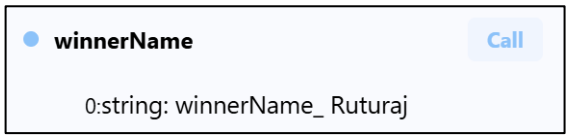


Fig 16 (left): winningProposal returns index 1. *Fig 17 (right): winnerName returns Ruturaj as a string.*

Final State of the Ballot:

Index	Proposal	Vote Count
0	Sandy	0
1	Ruturaj	1
2	Sumant	0

Account	Weight	Voted	Delegate	Vote
Account 1 (Chairperson)	1	false	none	-
Account 2	1	true	none	1 (Ruturaj)
Account 3	1	true	Account 4	-
Account 4	2	false	none	not yet cast

require Check Triggered	Revert Reason	Result
Account 2 voting a second time	"Already voted."	Reverted

require Check Triggered	Revert Reason	Result
Granting rights to an address that already has them	"Voter already has the right to vote."	Reverted

Observations:

1. **The bytes32 to string change worked end to end.** Proposal names were supplied as readable text at deployment, returned as readable text by the proposals getter, and returned by winnerName as "Ruturaj". No hexadecimal conversion was needed at any point, which is exactly the usability gain the modification was intended to provide.
2. **require guards reverted invalid transactions without side effects.** The repeated vote from Account 2 and the duplicate grant of voting rights were both rejected with their specified messages. Ruturaj's vote count remained at 1 after the failed second vote, confirming that the revert rolled back every change rather than partially applying the transaction.
3. **Delegation to a voter who has not yet voted transfers weight, not votes.** After Account 3 delegated to Account 4, Account 4's weight rose from 1 to 2 while every proposal's vote count stayed unchanged. The delegated vote is held in reserve and will only be counted when Account 4 actually votes, at which point that single vote will add 2 to the chosen proposal.
4. **Delegation is recorded as having voted.** Account 3 shows voted = true even though it never called vote. The contract deliberately marks a delegator as having voted, which prevents the same address from delegating and then also voting directly, a form of double voting that would otherwise be possible.
5. **The result reflects only votes actually cast.** Ruturaj won with a single vote while three units of voting weight remained uncast: Account 1 (the chairperson) never voted, and Account 4 holds a combined weight of 2 that has not been used. Had Account 4 voted for Sandy, the outcome would have changed to Sandy with 2 votes, which shows why a real election would need a defined closing point before winnerName is treated as final.
6. **Reading state is free, changing it costs gas.** Calls to chairperson, proposals, voters, winningProposal and winnerName appear as CALL entries in the log and create no transaction, because view functions only read state. Operations such as giveRightToVote, vote and delegate appear as mined transactions because they modify storage.
7. **Mapping defaults make permission checks simple.** Every address exists in the voters mapping with a zero weight by default, so an address that was never granted rights automatically fails the check require(sender.weight != 0). No explicit registration list is needed to reject unauthorised voters.

Conclusion:

In this experiment the Ballot smart contract was studied, modified and deployed using Solidity and the Remix IDE. The proposal name type was changed from bytes32 to string across the Proposal struct, the constructor and the winnerName function, allowing proposals to be supplied and returned as readable text. The contract was compiled with Solidity 0.8.34 and deployed on the Remix VM from Account 1, which became the chairperson, with three proposals named Sandy, Ruturaj and Sumant.

The complete voting workflow was then exercised across four accounts. Voting rights were granted by the chairperson, Account 2 voted for Ruturaj, and Account 3 delegated its vote to Account 4, raising Account 4's weight from 1 to 2. The require statements were verified directly: a repeated vote was rejected with "Already voted." and a duplicate grant of voting rights was rejected with "Voter already

has the right to vote.", in both cases reverting all state changes. The contract declared Ruturaj the winner at proposal index 1.

The experiment demonstrated the role of require statements in enforcing security and correctness, the use of mapping for address-based lookup, and the practical distinction between storage references and temporary memory data. It also illustrated the trade-off between bytes32 and string, where string improves readability at the cost of higher gas, and showed that delegation to a voter who has not yet voted transfers weight rather than votes, so the final tally depends on every delegate eventually casting their vote.